

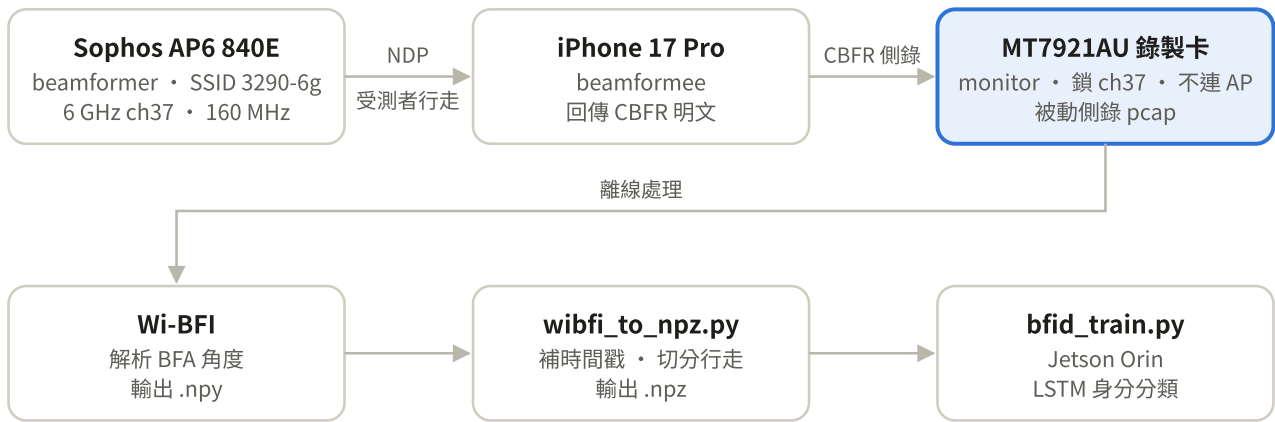
BFId 復刻建置指南 — BFI 身分推論攻擊實驗

Sep 28, 2026 更新 Sep 29, 2026 @Nelson

總覽

復刻目標是論文的 BFI 那條線：被動側錄 beamformee 回傳的明文 CBFR 幀，從中還原波束成形角度，訓練 LSTM 辨識走過 Wi-Fi 場域的人是誰。CSI 那條線當基準線，選配。

BFId 以明文回傳，單一錄製節點即可側錄



資料流 • 6 個節點，擷取與處理兩段

CBFR 實際上是 iPhone 回傳給 AP 的，錄製節點（MT7921AU / MT7925 USB 卡）只是在旁邊聽到——這正是論文說 BFI 威脅大於 CSI 的原因：監聽者不必在特定位置，也不需要網路密碼。

裝置	角色	頻段 / 模式	備註
Sophos AP6 840E	beamformer	6 GHz ch37 • 160 MHz • 11ax	需鎖通道與頻寬，關閉 11be 與 MLO
iPhone 17 Pro	beamformee	連上 AP 的 6 GHz SSID	需關閉私密 Wi-Fi 位址
MT7921AU / MT7925 USB	recording node	monitor mode，鎖 ch37 • 160 MHz	不連 AP；bfid.sh 的 MODE=recorder
Intel AX210	beamformee（選 用）/ 除錯	連 3290-6g	論文 perspective 同款；MODE=shared 可除錯， 另可跑 PicoScenes 取 CSI
x86 Linux 主機	解析	—	跑 Wi-BFI，conda 環境釘死 linux-64
Jetson Orin	訓練	—	跑 bfid_train.py，不適合跑 Wi-BFI
Raspberry Pi 4	備援 / CSI	5 GHz	Nexmon CSI，或第二個 perspective

受測者本身不需要攜帶任何裝置。

以裝置為單位的建置清單

先說結論：**真正要裝軟體的只有兩台**——插錄製卡的 x86 主機、以及 Jetson。AP、iPhone 都只是設定，沒有程式要裝。Pi 4 與 ESP32 是選用。

1. x86 Linux 主機（錄製卡與 AX210 插這台）

整個計畫的核心：擷取 + 解析。也是唯一需要 root 反覆操作的機器。

硬體與系統

- ☐ 一個 USB 3 埠給 MT7921AU / MT7925 錄製卡（`mt7921u` / `mt7925u` 驅動）
- ☐ 一個實體 M.2 / PCIe 插槽給 AX210（AX210 本身不要用 USB 轉接，monitor mode 會出問題）
- ☐ Ubuntu——BFI 這條路較新版本反而好（24.10 的核心 6.11 對 6 GHz monitor mode 有利）；但 PicoScenes 只支援 22.04，要做 CSI 基準線得另一台
- ☐ 核心 6.1 以上，6 GHz monitor mode 才穩
- ☐ 有線網路接到 AP 的 LAN（要從這台對 iPhone 灌 iperf3）

系統套件

```
sudo apt-get install tshark wireshark aircrack-ng iw iperf3 \
    linux-firmware wireless-regdb git
```

Wi-BFI 環境

- ☐ Miniconda 或 Anaconda
- ☐ `git clone https://github.com/kfoysalhaque/Wi-BFI.git`
- ☐ `conda env create -f wi-bfi.yml`，建不起來就手建 Python 3.10 + `pyshark==0.6 numpy matplotlib lxml`

本專案的腳本

- ☐ `wibfi_to_npz.py` 放在 Wi-BFI 旁邊（不需要 PyTorch，只要 numpy）

驗收

```
iw dev mon0 info          # type monitor, width 160 MHz
python main.py --help      # Wi-BFI 能跑
python wibfi_to_npz.py --help # 轉換工具能跑
```

⚠ PicoScenes 與 monitor mode 的衝突

若你要同時做 CSI 基準線，**PicoScenes 安裝時會強制把系統更新到最新核心**，而且它的驅動模組是按核心版本預編譯的（`picoscenes-driver-modules-<kernel>-generic`）。這會把你已經驗證好的 monitor mode 環境推到新核心上，可能就壞了。

兩個選擇：

1. **分開兩台機器**（或雙開機）——一台專跑 BFI monitor mode，一台專跑 PicoScenes CSI。你反正只有一張 AX210，本來就要分兩次錄。
2. **先裝 PicoScenes 再驗證 monitor mode**——讓核心先被推到最新，再在那個核心上調 monitor mode。

先只做 BFI 的話，**現在別裝 PicoScenes**。

2. Jetson Orin

只跑訓練，不碰擷取也不碰解析。

- ☐ 用 JetPack 提供的 PyTorch（NVIDIA 的 wheel），**不要 `pip install torch`**——PyPI 的 wheel 沒有 Jetson 的 CUDA 支援
- ☐ numpy
- ☐ `bfid_train.py` 放這裡
- ☐ **不要在這裡裝 Wi-BFI**——`wi-bfi.yml` 的 conda 相依是 linux-64 釘死版本，arm64 建不起來

驗收

```
python3 -c "import torch; print(torch.__version__, torch.cuda.is_available())"
python3 bfid_train.py --synthetic --repeats 1 --epochs 6
```

第二行不需要任何真實資料，準確率應該很快衝到 100%。沒衝上去就是環境問題，不是資料問題。

3. Sophos AP6 840E

沒有程式要裝，全部是設定。

- ☐ 韌體更到最新
- ☐ 確認管理介面（本機或 Sophos Central）改得到進階無線設定——**這一項決定這台 AP 能不能用**
- ☐ 按「Sophos AP6 840E 設定檢查清單」那節跑完 12 項
- ☐ PoE++ 供電（802.3bt）

4. iPhone 17 Pro

- ☐ 關閉「私密 Wi-Fi 位址」，記下真實 MAC
- ☐ 裝一個 iPerf3 App（當 server，讓 x86 主機從有線端灌進來）
- ☐ 接電，自動鎖定設成永不，關低功率模式
- ☐ 確認連的是 6 GHz SSID `3290-6g`

5. Raspberry Pi 4（選用）

當 CSI 基準線，或日後當第二個 perspective 的監聽節點。

- ☐ Pi OS Bookworm
- ☐ `nexmon_csi`——近期核心用它們的 `Makefile.rpi`，不再需要改過的 `brcmfmac` 驅動
- ☐ `nexutil`

內建 BCM43455c0，只能 5 GHz。**做不了 6 GHz，也做不了 BFI 的主實驗。**

6. ESP32 NodeMCU-32S（選用，最低優先）

只能 2.4 GHz / 802.11n / 64 子載波，**做不了 BFI**。價值在於十分鐘就能跑出一條 CSI，建立「無線感測真的有效」的信心。

關於你提到的 Arduino IDE：Arduino 的 ESP32 核心本身就是建在 ESP-IDF 上，所以 `esp_wifi_set_csi_rx_cb()` 這類函式可以直接在 sketch 裡呼叫。但現成的工具鏈（StevenMHernandez/ESP32-CSI-Tool、espressif/esp-csi）都是 ESP-IDF 專案，照著跑比自己在 Arduino 裡重寫省事很多。

- ☐ ESP-IDF（建議）或 Arduino IDE + ESP32 核心
- ☐ 兩張板：一張發送、一張接收

建置順序

1. **x86 主機 + 錄製卡的 monitor mode**—— `MODE=recorder ./bfid.sh up`，確認能鎖 6 GHz ch37
2. **AP 設定**——鎖通道、關 11be
3. **iPhone 設定**——關私密位址
4. **關卡驗證**——用 tshark 確認真的抓到 CBFR。這一關過不了就不要往下裝任何東西
5. **Wi-BFI 環境**
6. **Jetson 訓練環境**——可以跟 1~5 平行做，互不影響
7. （選用）Pi 4 或 ESP32 的 CSI 基準線

與論文的落差

四個落差必須先接受，它們不是做錯了，而是硬體與法規的張力。

落差	論文	你的環境	影響與因應
6 GHz 通道數	ch37 與 ch85 兩條	兩條都可用	ch85 的 160 MHz 區塊是 6265–6425，落在台灣 5945–6425 內。歐盟上限與台灣相同，論文作者在 KIT 就是這樣用的
CSI 與 BFI 錄製	兩台 AP 同時錄	需第二台 AP	通道不是限制，但 CSI 與 BFI 需要不同流量型態（200 Mb/s TCP vs 30 Kb/s UDP），所以論文用兩台 AP。只有一台就分兩次錄
BFI 特徵維度	740（74 × 10）	Wi-BFI 實測遠高於此	見下表。訓練腳本吃任意維度，但記憶體與訓練時間會升高
樣本規模	197 人	你能招到的人數	身分數越少準確率越高，數字不能與論文直接對比

特徵維度的實測值

這是按 Wi-BFI 原碼的子載波索引推算出來的，不是估計值。維度 = 子載波數 × 每子載波角度數（4x2 為 10 個，4x1 為 6 個）。

標準	頻寬	子載波數	4x2 特徵數	4x1 特徵數
AX	160 MHz	500	5000	3000

標準	頻寬	子載波數	4x2 特徵數	4x1 特徵數
AX	80 MHz	250	2500	1500
AX	40 MHz	122	1220	732
AX	20 MHz	64	640	384
AC	80 MHz	234	2340	1404
AC	40 MHz	108	1080	648
AC	20 MHz	52	520	312

你的目標設定 (AX / 160 MHz / 4x2) 會得到 **5000 維**，是論文 740 維的近七倍。論文的 74 個 channel 明顯對應更粗的子載波分組 (grouping, Ng)。若要接近論文維度，用轉換工具的 `--subsample` 對子載波等距抽樣 ($500 \div 7 \approx 71$ ，接近 74)。

★ 裝置角色：perspective 不是監聽者

論文 5.1.2 節明言：

For BFI, all perspectives are connected to the WiFi network established by the access point and receive traffic from it. This traffic triggers the channel sounding procedure regularly. All beamforming reports are recorded centrally by an additional beamforming recording node which is not part of the network.

所以 BFI 實驗裡有**兩種不同角色的裝置**，別搞混：

角色	是否連上 AP	做什麼	論文的硬體
Perspective (beamformee)	是，是正常客戶端	收流量 → 被 sounding → 送出自己的 CBFR	每個 perspective 一張 AX210，共四張
Recording node	否，完全在網路外	被動側錄所有 CBFR	另一個獨立節點

「多視角」的來源是**多個位置不同的客戶端**，不是多個監聽點。每個 perspective 的兩根互相正交的天線是**客戶端的收訊天線**。

這對你的硬體規劃有直接影響：**一張 AX210 不夠**，因為它同時只能扮演一個角色。最小可行組合：

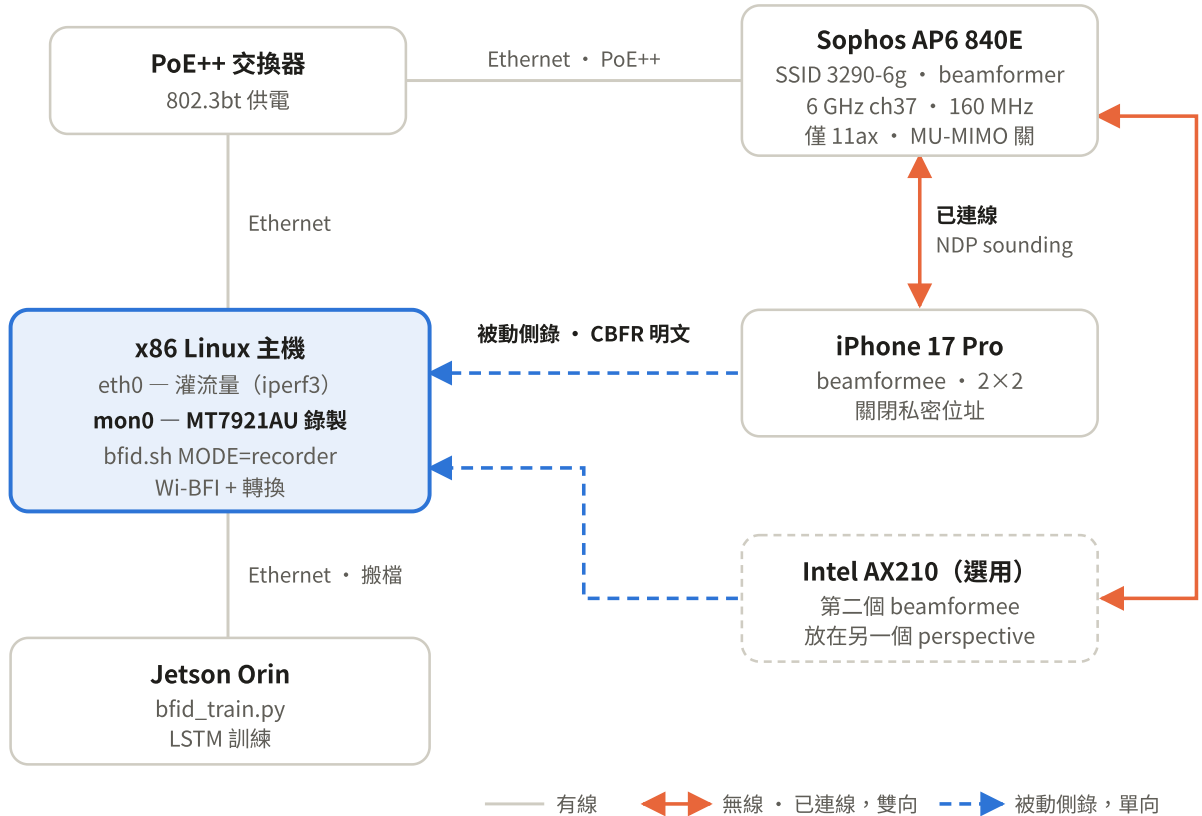
- AX210 (已有) 當 perspective，或直接用 iPhone 當唯一 beamformee
- 一張 MT7921AU / MT7925 USB 網卡當 recording node

iwlwifi 在 managed + monitor 並存下只遞送極少數幀 (實測 20 秒 24 個，且抓不到其他客戶端的單播)，做不了 recording node。

網路拓樸

側錄那條線是單向的，而且不需要連上網路

錄製卡不向 AP 驗證、不取得 IP，只是聽；AX210 空出來當第二個 beamformee



網路拓樸 • 6 個節點，3 種連結

三種連結要分清楚。**有線那三條只是在搬流量與檔案**：x86 主機從 `eth0` 對 iPhone 灌 iperf3（路徑是 `eth0` → 交換器 → AP → 無線 → iPhone），錄完再把 `.npz` 搬到 Jetson。

無線那兩條差很多。AP 與 iPhone 之間是正常的雙向連線，channel sounding 就在這條線上發生；錄製卡的 `mon0` 只是在旁邊聽，**單向、不驗證、沒有 IP、不在任何 AP 的客戶端名單上**。AX210 若也連上 3290-6g，它就是第二個 beamformee，它的 CBFR 同樣會被錄製卡聽到。

這張圖裡最值得注意的是：`mon0` 那條虛線不連到交換器，也不連到 AP。側錄者完全在網路之外，這就是論文把它稱為「攻擊」的原因。

一些常見的變形

- **沒有 PoE++ 交換器**：用 802.3bt 單口注入器，接一台普通交換器。AP6 840E 要 PoE++，一般 PoE+ 供電可能讓它降規或不開 6 GHz radio。
- **Jetson 不接網路**：用 USB 碟搬 `.npz` 也行，訓練本身不需要網路。
- **x86 主機只有一張網卡**：不行。`mon0` 必須專屬於錄製卡，而灌流量要另一個介面。用內建有線網卡做 `eth0`。

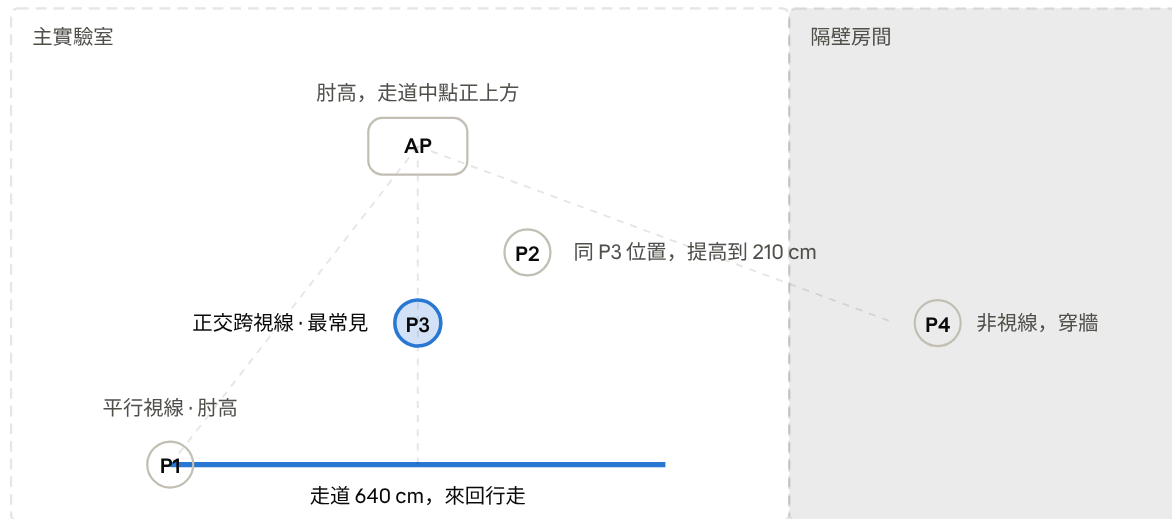
- **用 iPhone 自己往外灌流量**（跑測速 App）：可以，但流量型態不好控制，sounding 率比較不穩。iperf3 固定 bitrate 可重現性好很多。

場地佈置

先只做 P3。四個 perspective 同時錄需要四套天線與四張 AX210，你手上只有一張。先用最常見的 P3（走道正交穿越視線）把整條流程跑通，再考慮擴充。

四個 perspective 分別測試平行、高度、正交與非視線

俯瞰平面，對應論文 Figure 2；虛線為牆面



場地平面 • 1 條走道，4 個 perspective

除了 P2 以外，所有天線與 AP 都架在肘高或略高，因為論文認為這個高度能捕捉到最多步態資訊。P2 刻意拉高到 210 cm，就是用來測高度的影響。

每個 perspective 在論文裡都是一組兩根天線（一根 BFI、一根 CSI），兩根互相正交，一根朝上、一根垂直於到 AP 的視線。

擴充到多 perspective 的順序

1. **P3 先跑通**——這是相關研究最常用的角度，有問題最容易判斷是設定錯還是環境問題。
2. **加 P1（平行視線）**——論文發現 P1 平均單一 perspective 準確率最高，跨 perspective 實驗也是拿它當訓練集。
3. **加 P4（非視線）**——穿牆場景，隔壁房間。這是隱私威脅最具說服力的一個。
4. **加 P2（高位）**——與 P3 同位置不同高度，只是為了隔離出高度這個變因。

BFI 有一個 CSI 沒有的便宜捷徑：多 perspective 的 BFI 只需要一個監聽節點，因為所有 beamformee 都在廣播 CBFR。你只要放幾台不同位置的 client（舊手機、筆電）就能拿到多個角度。CSI 則必須每個角度都放一張網卡。

Sophos AP6 840E 設定檢查清單

先讀這段：**AP6 系列無法用 Sophos 防火牆管理**，走的是 Sophos Central 雲端管理，單台另有本機管理選項。雲端介面未必給你鎖通道、鎖頻寬這種細粒度控制。**這是整個計畫風險最高的單一項目，先確認它。**

如果兩種管理介面都給不了固定通道與 160 MHz，這台 AP 就不適合做這個實驗——改用一台可完全手動設定的 Wi-Fi 6E AP（論文用 TP-Link Archer BE800）。

#	設定項目	目標值	為什麼	怎麼確認
1	管理方式	本機管理或 Central，能改無線進階設定	沒有細粒度控制就做不下去	能不能手動指定通道號碼
2	6 GHz radio	開啟	三頻機預設未必開 6 GHz	客戶端掃得到 6 GHz SSID
3	國別 / 法規域	台灣	決定 6 GHz 可用通道	通道清單只到 ch≈93 以下
4	通道	固定 ch37	自動通道會在錄製中途跳走	不是 Auto，是寫死的數字
5	頻寬	160 MHz	子載波數決定特徵維度	不是 Auto；若只能 80 MHz 也可接受
6	無線標準	僅 802.11ax	Wi-BFI 只支援 AC 與 AX	關閉 11be / Wi-Fi 7
7	MLO（多連結）	關閉	多連結會讓 sounding 分散到別的頻段	設定裡找 Multi-Link Operation
8	Band steering	關閉	避免 iPhone 被踢回 5 GHz	通常在 SSID 進階設定
9	從屬 SSID	3290-6g，6 GHz 專用，單一頻段	同名多頻段會讓你分不清客戶端在哪	AP 上只有 6 GHz radio 廣播這個名稱
10	安全性	WPA3（6 GHz 強制）	6 GHz 不允許 WPA2	連得上就對了
11	Beamforming	開啟	沒有它就沒有 CBFR	通常預設開，若有開關別關掉
12	DFS / 自動重選	關閉	錄製中途換通道等於資料毀了	錄完一輪再檢查通道沒變

關於 11be

這項客戶端也要配合。iPhone 17 Pro 支援 Wi-Fi 7，若 AP 允許 11be，兩者會協商成 EHT，CBFR 幀的格式跟著變成 EHT 版本——**Wi-BFI 解不出來**，而且未必會報錯，可能只是給你一堆垃圾數值。

若 AP 沒有「僅 11ax」選項，退而求其次：改用 5 GHz + 802.11ac + 80 MHz。離論文遠一點，但 Wi-BFI 對 AC 的支援反而更成熟，整條流程能先跑通。

第 13 項：MU-MIMO 請關閉

前面清單漏了這項。main.py 開頭就寫了 mu-mimo is not available for AX yet，但它只印訊息、不中止。而且 AX 路徑的過濾條件是寫死的：

```
# AX : 寫死 SU
display_filter='wlan.he.mimo.feedback_type==SU && wlan.addr==%s' % (MAC)

# AC : 會帶入參數
display_filter='wlan.vht.mimo_control.feedbacktype==%s && wlan.addr==%s' % (mimo, MAC)
```

所以對 AX 傳 MU 會用 MU 的位元寬度去解析 SU 的幀——不報錯，解出來是垃圾。沉默失敗，務必避開。

實務影響視客戶端數量而定：

情境	MU-MIMO 開著會怎樣
只有一個 beamformee	AP 幾乎不會做 MU sounding，實質無差別
論文架構（四個 perspective 同時關聯）	AP 可能分組做 MU sounding，那些 CBFR 會被 ==SU 過濾掉， 有效取樣率下降

順帶一提：MU 的量化其實更細（SU 是 `psi_bit` 2 或 4、`phi_bit` 4 或 6；MU 是 5 或 7、7 或 9），角度數量由天線組態決定所以維度一樣，但解析度更高。**Wi-BFI 對 AC 是支援 MU 的**，所以若最後走 5 GHz / 11ac 路線，MU 反而變成可用選項，特徵品質可能更好。這在論文裡沒有探討。

SSID 為什麼要單獨開一個

一個 radio 一次只能在一個通道上，這是標準本身的限制。但同一個 SSID 可以同時廣播在 2.4 / 5 / 6 GHz（同名、不同 BSSID、不同通道），而這會出事：

- **Band steering** 會把 beamformee 推到別的頻段，錄到一半資料就斷了
- `mon0` 是繼承 `wlp1s0` 的 **channel context**，SSID 跨頻段會讓這個綁定變得不可預測

所以開一個 **6 GHz 專用、名稱帶後綴的 SSID**：`3290-6g`，與日常網路分開。這不是技術限制，是實驗控制。

原本的測試 SSID `!!3290-1` 已停用：開頭的 `!` 在 `bash` 互動模式會觸發歷史展開，寫進指令或 `nmcli` 參數時容易出錯。`bfid.sh` 的預設值已改為 `3290-6g`。改名時記得三個地方一起動：

- AP 上把 6 GHz SSID 改名為 `3290-6g`
- iPhone 忘記舊網路、重新連 `3290-6g`，並再次確認「私密 Wi-Fi 位址」是關的（每個網路各自設定）
- 若 `~/bfid.conf` 裡有 `SSID=` 那一行，改掉或刪掉；`bfid.sh up` 發現既有連線 `bfid-6g` 綁的是舊名稱時會自動更新

iPhone 17 Pro 作為 beamformee

四個設定，漏了第一個就拿不到連續序列。

1. **關閉「私密 Wi-Fi 位址」**。設定 → Wi-Fi → 點這個網路旁的 ⓘ → 關閉「私密 Wi-Fi 位址」。Wi-BFI 是用 MAC 篩選 beamformee，MAC 隨機化會讓你的序列中途斷掉。關掉後在同一頁面記下「Wi-Fi 位址」，那就是後續所有指令要用的 MAC。
2. **確認真的連在 6 GHz 的 `3290-6g`**。同一頁面的連線資訊會顯示頻段。若連到 5 GHz，回頭檢查 AP 的 band steering 與 SSID 設定。

3. **錄製時保持螢幕亮著**。螢幕關閉進入省電後，channel sounding 的頻率會明顯下降，sounding 率掉下來序列就變短。插電並把自動鎖定設成永不。
4. **關閉低功率模式與 Wi-Fi 輔助功能**。這些都會影響客戶端參與 sounding 的積極度。

天線組態

iPhone 是 2x2，配 4x4 的 AP840E 剛好是 Wi-BFI 的 4x2，與論文一致。但實際協商出來的空間串流數要以抓到的封包為準——若 Wi-BFI 解出來的維度與預期對不上，改試 4x1。

另一個選項

若 iPhone 的 11be 關不掉（AP 端沒有選項），拿一台舊一點的 Wi-Fi 6 手機或筆電當 beamformee 會簡單很多。受測者不需要攜帶這台裝置，它只是放在定點的訊號來源。

Monitor mode 建置

本節的指令以 AX210 為例；換成錄製卡時把介面名換成 USB 卡的（`iw dev` 查），`bfid.sh` 的 `MODE=recorder` 已包好整套流程。6 GHz monitor mode 的支援比 5 GHz 脆弱，核心與韌體版本都有影響。先把這段跑完再碰其他東西。

環境檢查

```
uname -r                # 核心 6.1 以上較安全
lspci | grep -i network  # 確認 AX210 被認到
dmesg | grep -i iwlwifi | head -20  # 韌體載入狀況
iw list | grep -A2 'Frequencies'    # 看得到 5955-6415 MHz 才有 6 GHz
```

若 `iw list` 根本沒列出 6 GHz 頻率，先更新 `linux-firmware` 與核心，再回來。

設定法規域

```
sudo iw reg set TW
iw reg get                # 確認 country TW, 且 5945-6425 MHz 有列出
```

6 GHz 在台灣只開放 **5945–6425 MHz**（NCC 2023-08-25 公告，與歐盟相同；20 MHz 通道為 ch1–ch93，160 MHz 區塊為 ch15 / ch47 / ch79）。法規域沒設對，下一步鎖通道會直接失敗。

建立 monitor 介面

```
# 找出 phy 名稱
PHY=$(iw dev wlan0 info | awk '/wiphy/ {print "phy"$2}')

# 建立專用 monitor 介面（不動原本的 wlan0）
sudo iw phy $PHY interface add mon0 type monitor
sudo ip link set mon0 up

# 鎖定 6 GHz ch37, 160 MHz
# 6 GHz 中心頻率 = 5950 + 5 * 通道號, ch37 -> 6135 MHz
# 160 MHz 的中心在 6185 MHz (ch47, 涵蓋 6105-6265)
sudo iw dev mon0 set freq 6135 160 6185

# 確認
iw dev mon0 info
```

最後一行要看到 `type monitor`、`channel ... width: 160 MHz`。這兩項任一不對就不要往下走。

⚠ 要不要對 mon0 下 `set freq`，取決於哪張卡在聽

上面的 `set freq` 只適用於獨立錄製卡。兩種情境的做法相反，`bfid.sh` 用 `MODE` 區分：

MODE	誰在聽	是否關聯 AP	set freq	用途
shared	AX210 同一張卡	是，wlp1s0 連上 3290-6g	不要下，mon0 跟著 wlp1s0 的 頻道走	只適合除錯
recorder	MT7921AU / MT7925 USB 卡	否，完全在網路外	必須下，直接鎖 ch37 / 160 MHz	正式錄製（論文架構）

`shared` 不能下 `set freq` 的原因：iwlwifi 是 `real-rtm` 驅動，monitor 介面沒有自己的 channel context 時封包會被靜默丟棄。但即使繞過這點，iwlwifi 在 `managed + monitor` 並存下遞送的幀依然極少，所以正式錄製要換 `recorder`：

```
# ~/.bfid.conf
MODE=recorder
REC_IF=wlx00c0cafe1234      # 用 iw dev 查 USB 卡的實際介面名
```

```
./bfid.sh doctor      # 會檢查驅動不是 iwlwifi、頻道落在台灣範圍、區塊對齊
./bfid.sh up          # 停用 REC_IF、建 mon0、set freq 6135 160 6185
./bfid.sh verify
```

`recorder` 模式下 AX210 就空出來了，可以當第二個 beamformee（論文的 perspective 就是 AX210）。

i 6 GHz 的中心頻率怎麼算

20 MHz 通道中心 = $5950 + 5 \times \text{通道號}$ 。但 80 / 160 MHz 要填的是**所屬區塊**的中心，不是通道本身的頻率。區塊邊界從 5945 MHz 起算：

頻寬	包含 ch37 的區塊	要填的中心
80 MHz	6105–6185	6145 (ch39)
160 MHz	6105–6265	6185 (ch47)

填錯會直接失敗。換其他通道時記得重算：160 MHz 的中心通道是 15 / 47 / 79 / 111…，80 MHz 是 7 / 23 / 39 / 55…

⚠ Intel 網卡是 self-managed，iw reg set 對它無效

`iw reg get` 會同時列出 `global` 與 `phy#0`。Intel 的卡是自管式法規域，法規來自韌體，**不吃 `iw reg set TW`**：

```
global
country TW: DFS-FCC           ← 你設的
phy#0 (self-managed)
country US: DFS-UNSET         ← AX210 實際採用的
```

實務上不影響 ch37 (US 比 TW 寬鬆，6135 MHz 兩邊都允許)，但有兩件事要自律：

- 卡片會列出到 ch109 以上 (美國範圍)，**台灣只到 ch93 (6415 MHz)**。不要用超出的通道。
- 台灣 6 GHz 標示 `NO-OUTDOOR`，實驗留在室內。

關於 (no IR)

6 GHz 通道都帶 (no IR) (No Initiate Radiation)，表示不能主動發射，必須先偵測到 AP 訊號。**對 monitor mode 沒影響**——純被動接收本來就不發射。它擋的是當 AP 或主動送 probe request 那類行為。

若 160 MHz 設不起來

先退到 80 MHz 確認整條鐵路通了，再回頭攻 160：

```
sudo iw dev mon0 set freq 6135 80 6145
```

80 MHz 的 BFI 特徵維度是 2500 (對 4x2)，依然足夠做身分辨識。論文自己也說 6 GHz 與 5 GHz 的差異應該很小。

常見問題

NetworkManager 會跑來把介面改回去。錄製前先讓它放手：

```
sudo nmcli device set mon0 managed no
```

關卡驗證：真的抓到 CBFR 了嗎

這是整份文件最重要的一步。這一關過不了，後面裝 Wi-BFI、接受測者、訓練模型全部是白工。花十分鐘在這裡，省掉後面兩天。

準備

讓 iPhone 連上 AP，產生持續流量。在 iPhone 上跑一個測速 App，或從有線端機器跑 iperf3 對它灌：

```
# 在有線端機器上 (iPhone 裝 iperf3 App 當 server)
iperf3 -c <iPhone_IP> -t 60 -b 200M
```

沒有流量就沒有 channel sounding，也就沒有 CBFR。論文對 BFI 用的是 200 Mb/s TCP。

抓一段

```
sudo timeout 30 tcpdump -i mon0 -w /tmp/probe.pcap
```

判讀

兩個標準的過濾條件不同，這是 Wi-BFI 原碼裡用的那兩行：

```
# 802.11ax
tshark -r /tmp/probe.pcap -Y 'wlan.he.mimo.feedback_type==SU' | head

# 802.11ac
tshark -r /tmp/probe.pcap -Y 'wlan.vht.mimo_control.feedbacktype==SU' | head

# 統計有幾幀，並確認發送端是 iPhone
tshark -r /tmp/probe.pcap -Y 'wlan.he.mimo.feedback_type==SU' \
  -T fields -e wlan.sa -e wlan.da | sort | uniq -c
```

最後一行應該看到一堆「iPhone 的 MAC → AP 的 MAC」。那就是 CBFR。

量到底多快

```
tshark -r /tmp/probe.pcap -Y 'wlan.he.mimo.feedback_type==SU' \
  -T fields -e frame.time_epoch | \
  awk 'NR==1{a=$1} {b=$1; n++;} END{printf "%d 幀 / %.1f 秒 = %.1f Hz\n", n, b-a, (n-1)/(b-a)}'
```

論文調教到的 BFI 取樣率約 10 Hz。你拿到 5 Hz 以上就可以開始。論文 6.1 節發現 BFI 對低取樣率相當穩健，平均只剩幾個封包時準確率才明顯下滑。

抓不到的話

不要繼續往下。跳到最後的排查分支那節。

Wi-BFI 安裝與解析

裝在 **x86 Linux 主機**，不是 **Jetson**。 `wi-bfi.yml` 的 conda 相依是 linux-64 釘死版本（build hash 如 `h06a4308`），arm64 建不起來。

系統套件

```
sudo apt-get install tshark wireshark aircrack-ng
```

pyshark 是 tshark 的包裝層，**tshark 版本會直接影響解析結果**，這是最常見的踩雷點。

取得並建置

```
git clone https://github.com/kfoysalhaque/Wi-BFI.git
cd Wi-BFI
conda env create -f wi-bfi.yml
conda activate wi-bfi
```

若 conda 環境建不起來

先說：離線解析根本用不到 conda。那份 yml 是 2023 年的，`pysimplegui` 後來授權模式有變動。離線解析根本用不到 GUI 那組，直接手建一個就好：

```
# 先看 Python 版本。Ubuntu 22.04 內建就是 3.10，正好是 Wi-BFI 要的版本
python3 --version

# 是 3.10.x 的話，直接用 venv，不需要 conda
sudo apt-get install -y python3-venv python3-pip
python3 -m venv ~/wi-bfi-venv
source ~/wi-bfi-venv/bin/activate
pip install pyshark==0.6 numpy matplotlib lxml
```

不是 3.10.x 才需要 Miniconda：

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
# 安裝過程會問 "update your shell profile to automatically
# initialize conda?" — 回 yes, 回 no 每次開 shell 都要手動 source

# 裝完找不到 conda 指令的話, 手動初始化
which conda || ~/miniconda3/bin/conda init bash
source ~/.bashrc

conda create -n wi-bfi python=3.10 -y
conda activate wi-bfi
pip install pyshark==0.6 numpy matplotlib lxml
```

成功的話提示字元會變成 (wi-bfi) nelson@...。之後每開一個新 shell 都要重跑 `conda activate wi-bfi`，否則會用到 base 環境的 Python。

兩個常見的坑：`conda create -n python=3.10` 會建立一個叫 `python=3.10` 的空環境——`-n` 後面要先接環境名稱。另外解析不需要 root，用 root 建環境會裝到 `/root/` 底下，之後換一般使用者跑會找不到。只有 `tcpdump`、`iw` 這些才需要 root。

完整相依清單（供參考）：Python 3.10.0、`pyshark==0.6`、`numpy==1.26.0`、`matplotlib==3.8.0`、`lxml==4.9.3`，加上即時繪圖用的 `pyqt5`、`pyqt6`、`pysimplegui`、`cairocffi`。

支援範圍

參數	可選值
standard	AC / AX
mimo	SU / MU（MU 尚不支援 AX）
天線組態	4x2 / 4x1 / 3x3 / 3x2 / 3x1
頻寬	160（僅 AX） / 80 / 40 / 20

你的組合 **AX + SU + 4x2 + 160** 落在支援範圍內。

解析

```
python main.py <pcap檔名> AX SU 4x2 160 <iPhone的MAC> <封包數> <V矩陣輸出> <BFA輸出>
```

實例：

```
python main.py subject_001_normal.pcap AX SU 4x2 160 \
    aa:bb:cc:dd:ee:ff 2000 V_s001_normal bfa_s001_normal
```

封包數要小於等於 pcap 裡該 **MAC** 符合條件的封包數，否則會在迴圈裡爆掉。main.py 的過濾條件含 wlan.addr==MAC，所以計數也要加同一個條件，不能只數全部 CBFR：

```
tshark -r subject_001_normal.pcap \
-Y 'wlan.he.mimo.feedback_type==SU && wlan.addr==aa:bb:cc:dd:ee:ff' | wc -l
```

bfid.sh capture 印出的 main.py 指令已經用這個數字。

大量檔案用 main_extract_batch.py —— pyshark 逐檔解析很慢，197 人規模的資料用 main.py 一檔一檔跑會跑到天荒地老。

輸出是什麼

main.py 最後只做兩件事：

```
np.save(saved_vmatrices, v_matrices_all)
np.save(saved_angles, bfi_angles_all_packets)
```

角度檔是一個 .npy，形狀（封包數，子載波數，角度數）。注意它沒有保留任何時間戳，也沒有切分成一次次行走。這兩個缺口由下一節的轉換工具補上。

錄製流程

流量產生

論文是用參數最佳化跑出來的設定：BFI 用 200 Mb/s TCP，CSI 用 30 Kb/s UDP（兩者目標相反——BFI 要灌滿讓 AP 頻繁發起 sounding，CSI 反而不需要）。

```
# 整個錄製期間持續跑著
iperf3 -c <iPhone_IP> -t 3600 -b 200M
```

錄製前先確認 sounding 率（上一節那段 awk）足夠，再請受測者進場。

每位受測者的項目

行走風格	來回次數	用途
正常行走	20	主實驗的訓練與測試集
背背包	10	跨風格測試（論文 6.3）
抬飲料箱	10	跨風格測試
穿越閃門	10	跨風格測試
快走	10	跨風格測試

論文的受測者平均年齡 23.2 歲，每人參與約一小時。若你只想先驗證可行性，先做「正常行走 20 次」一種風格就好，每人約十分鐘。

服裝限制

論文要求不穿寬鬆衣物、裙子、洋裝、高跟鞋，目的是讓步態不被遮蔽。論文自己也承認這可能高估了 BFI 的辨識潛力，同時又讓受測者之間差異變小、辨識變難——兩個方向的偏差都存在。

空房間 baseline

別跳過這個。 每位受測者進場前錄 45 秒的空房間。這是論文 6.6.1 節的驗證實驗：拿訓練好的模型去預測空房間錄音，如果它能把空房間對到前後那位受測者，就表示模型學到的是「場次」而不是「人」。

論文的 BFI 模型在這個測試上 top-2 準確率只有 2.34%，低得很健康。你的數字若明顯偏高，代表你的模型在作弊。

命名規則

```
recordings/
  subject_001/
    empty_before.pcap
    normal.pcap
    backpack.pcap
    crate.pcap
    turnstile.pcap
    fast.pcap
  subject_002/
    ...
```

每種風格一個 pcap，裡面包含那種風格的所有來回。行走之間自然的停頓（轉身、等待）會在時間戳上留下空隔，轉換工具就是靠這個空隔自動切分。刻意在每次行走之間停滿二秒，切分會準很多。

錄製指令

```
sudo tcpdump -i mon0 -w recordings/subject_001/normal.pcap
```

資料轉換與訓練

兩支腳本：`wibfi_to_npz.py` 接在 Wi-BFI 後面補上時間戳與切分，`bfid_train.py` 跑論文的 LSTM。

轉換

先看資料長什麼樣，不寫檔：

```
python wibfi_to_npz.py --angles bfa_s001_normal.npy \
  --pcap recordings/subject_001/normal.pcap \
  --standard AX --mimo SU --mac aa:bb:cc:dd:ee:ff --inspect
```

它會報出維度、取樣率、切出幾個片段、每個片段多長。**先看片段數對不對**。正常行走 20 次來回應該切出接近 40 個片段（論文每人 40 條）。切太少就調低 `--gap`，切太多就調高。

確認後寫出：

```
python wibfi_to_npz.py --angles bfa_s001_normal.npy \
  --pcap recordings/subject_001/normal.pcap \
  --standard AX --mimo SU --mac aa:bb:cc:dd:ee:ff \
  --subject subject_001 --out-dir data/bfi_normal \
  --gap 1.5 --subsample 7
```

`--subsample 7` 把 160 MHz 的 500 個子載波降到 71，維度從 5000 降到 710，接近論文的 740。不加這個參數也能跑，只是慢很多。

每位受測者、每種風格都跑一次，風格分別收到不同 `--out-dir`（`data/bfi_normal`、`data/bfi_backpack`...），但 `--subject` 保持一致——跨風格實驗靠目錄名稱對應身分。

環境

Jetson Orin 上的 PyTorch 要用 NVIDIA 的 JetPack 版本，不是 `pip install torch`。裝完先確認：

```
python3 -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

然後用合成資料確認整條訓練管線通了，這不需要任何真實資料：

```
python bfid_train.py --synthetic --repeats 1 --epochs 6
```

四個實驗

論文節次	實驗	指令
6.1	主實驗：BFI 能辨識身分	<code>python bfid_train.py --data-root data/bfi_normal --repeats 5</code>
6.3	跨行走風格（不重新訓練）	<code>python bfid_train.py --data-root data/bfi_normal --eval-root data/bfi_backpack --repeats 5</code>
6.5	跨 perspective	同上， <code>--eval-root</code> 改指另一個 perspective 的目錄
6.6.1	空房間驗證（top-2）	<code>python bfid_train.py --data-root data/bfi_normal --eval-root data/bfi_empty --topk 2</code>

論文節次	實驗	指令
附錄 B	資料切分比例影響	反覆調 <code>--test-ratio 0.8 / 0.6 / 0.4 / 0.2</code>

模型架構

腳本照論文 4.3 節實作：標準 LSTM → 兩層 FC（各含 BatchNorm + ReLU）→ softmax，ADAM 最佳化，除了特徵標準化不做任何前處理。80:20 切分，重複五次獨立切分後回報 mean ± std。

兩個實作細節值得知道：標準化的統計量只用訓練集計算，避免資料洩漏；變長序列用 `pack_padded_sequence`，所以 padding 不會污染 LSTM 的最終狀態（這點已實測驗證：同一條序列單獨跑與整批跑，輸出差異 2.2e-08）。

判讀結果

腳本會同時印出隨機猜中率。準確率要與它對比，不是與論文的 99.5% 對比——你的身分數少很多，數字天然就會漂亮。真正有意義的比較是空房間驗證與跨風格測試。

失敗排查分支

症狀：monitor mode 起不來

檢查	判斷	做法
<code>iw list</code> 沒列出 monitor	韌體或核心太舊	更新 <code>linux-firmware</code> 與核心
<code>interface add</code> 失敗	名稱衝突或權限	先 <code>iw dev mon0 del</code> 再重建
介面自己變回 managed	NetworkManager 接管	<code>nmcli device set mon0 managed no</code>

症狀：6 GHz 選不到通道

檢查	判斷	做法
<code>iw reg get</code> 不是 TW	法規域未設	<code>sudo iw reg set TW</code> ；若被覆蓋，改 <code>/etc/default/crda</code> 或 <code>wireless-regdb</code>
<code>iw list</code> 沒 5955–6415 MHz	韌體不支援 6 GHz	更新 <code>linux-firmware</code>
<code>set freq</code> 報 invalid argument	中心頻率算錯	160 MHz 的中心與主頻率不同，檢查第三個參數
仍然不行	該核心對 AX210 6 GHz monitor 支援不完整	退到 5 GHz + 802.11ac + 80 MHz

症狀：抓不到 CBFR 幀

這是最常發生的。依序排除：

1. **pcap 是不是空的？** `tcpdump -r probe.pcap | wc -l`。總封包數就是 0 → monitor 介面或通道不對，回上一節。
2. **有封包但沒 CBFR？** 先確認過濾條件的標準選對了——`wlan.he.*` 是 11ax，`wlan.vht.*` 是 11ac。兩個都試。
3. **兩個都沒？** 很可能協商成了 11be。回 AP 設定關 11be 與 MLO。用 `tshark -r probe.pcap -Y 'wlan.eht' | head` 看有沒有 EHT 幀來驗證這個猜測。
4. **確定是 11ax 但還是沒？** 流量不足。確認 iperf3 真的在跑，而且是對 iPhone 灌。
5. **有流量也還是沒？** AP 的 beamforming 可能被關掉了，或 iPhone 進了省電。把螢幕點亮再試。
6. **抓到了但發送端不是 iPhone？** MAC 隨機化沒關。回 iPhone 設定。

症狀：sounding 率過低

sounding 頻率完全取決於流量型態，這是論文自己在預備實驗裡觀察到的。往上調 bitrate、改用 TCP、確認 iPhone 沒進省電。

若實在拉不上去，**不用太焦慮**——論文 6.1 節發現 BFI 對低取樣率相當穩健，這也是它認為「降低 sounding 頻率」不是有效防禦手段的原因。

症狀：Wi-BFI 報錯或維度怪異

症狀	原因	做法
<code>StopIteration</code>	封包數參數大於 pcap 實際符合條件的封包數	先用 tshark 數清楚再填
解出來是一堆垃圾數值	協商成 11be，幀格式不對	關 11be
維度與預期差很多	天線組態或頻寬參數填錯	對照本文「特徵維度實測值」表反推正確參數
解析極慢	pyshark 包 tshark，本身就慢	用 <code>main_extract_batch.py</code>

症狀：轉換工具切不出片段

症狀	原因	做法
切出 1 個超長片段	<code>--gap</code> 太大	調低到 0.8~1.0
切出幾百個碎片	<code>--gap</code> 太小或 sounding 本身很疏	調高到 2~3，並提高 <code>--min-len</code>
時間戳數少於 BFA 數	兩者來自不同 pcap 或過濾條件	確認 <code>--mac</code> 與 <code>--standard</code> 與 Wi-BFI 跑的一致

症狀：訓練不收斂或接近隨機

1. **先跑 `--synthetic`**。合成資料跑得出高準確率 → 腳本沒問題，是資料問題。跑不出來 → 環境或 PyTorch 安裝問題。

2. **每人樣本數夠嗎？** 論文每人 40 條正常行走。低於 10 條很難學。
3. **序列是不是太短？** 論文 BFI 平均長度約 40。你切出來平均只有 5 步的話，sounding 率太低。
4. **是不是學到場次而不是人？** 跑空房間驗證。top-2 明顯高於隨機 → 模型在認場次，不是認人。這時要檢查是不是每人錄在不同時段而環境慢漂。
5. **調超參。** `--hidden 256 --fc 256`、`--lr 3e-4`、`--epochs 300 --patience 50`。論文是用 Optuna 對 CSI 與 BFI 分別調的。

倫理與法規

這個實驗收的是步態生物特徵，不是一般的網路封包。把它當成一般技術測試處理會出問題。

資料性質

論文的同意書明白寫明會錄製生物特徵資料，並指出那屬於 GDPR 第 9 條的特殊類別個人資料。台灣個資法也把醫療、基因、性生活、健康檢查、前科列為特種個資，步態生物特徵雖未明列，但可識別性很強。**具體適用與審查要求請諮詢貴單位的法務或研究倫理單位，這份文件不是法律意見。**

受測者同意

論文的作法值得照抄：

- 資料收集經機構倫理委員會核准，依赫爾辛基宣言執行
- 同意書明示會錄製生物特徵資料，不是藏在泛用條款裡
- 受測者可隨時拒答、隨時退出
- 資料是否分享給其他研究者是**分開的選項**（197 人中 181 人同意）
- 有報酬（論文給 15 歐，參與約一小時）

資料保管

論文選擇**不無條件公開**資料集，而是讓研究者申請取用。理由是資料裡的個人資訊量體大，而且未來還可能被推論出現在還不知道的東西。你的資料集也應該照這個標準：不上公開儲存空間、不進版本控制、存放位置與保存期限寫進同意書。

場地的第三人

這點比論文情境更重要：監聽是被動的，不需要網路密碼，也不會留下痕跡。如果你在辦公室做這個實驗，**確保錄製期間只有受測者在場**，並且你監聽的是自己架的 AP 與自己的 client，不是公司的生產網路。側錄公司網路上其他人的裝置——即使只是 CBFR——是完全不同的一件事。

為什麼這篇論文把它叫「攻擊」

論文特別指出，很多相關研究自稱「隱私保護」或「不造成隱私疑慮」，但作者們認為這就是對隱私的攻擊：Wi-Fi 無所不在、能穿牆感測、而且很容易在當事人不知情的情況下進行。你復刻這個實驗的目的若是理解威脅，這個框架值得一併復刻過來。